

LILITH Architecture & Research Master

Zero-to-hero canonical architecture, state ownership, safety model, roadmap, and complete RQ-001 through RQ-121 register.

DOCUMENT	REVISION	DATE
LAR-MASTER	1.0	14 September 2026

PURPOSE

A durable recovery point for the LILITH project: architecture, decisions, evidence, uncertainties, and next directions. Current facts and future intent are deliberately separated.

NAVIGATION

Contents

Contents	2
Executive summary	3
What LILITH is - and is not	4
Current implementation vs target architecture	5
Ownership, state, and continuity	6
Memory architecture	7
Goals, cognition, planning, and workers	8
Governance, verification, and security	9
Cross-device continuity, voice, and Hsin	10
Relationship and affective continuity	11
Roadmap and architecture invariants	13
Complete research-question register	14
Source and interpretation notes	27
LILITH in plain English	28

ORIENTATION

Executive summary

LILITH is a persistent, governed personal AI operating system. Its defining continuity lives above any single model, runtime, interface, database, device, body, or cloud. The project asks how one evolving intelligence can remember, plan, act, learn, coordinate tools, gain bounded authority, and remain honest and auditable over years.

IDENTITY INVARIANT

Replace the model, Hermes runtime, UI, database implementation, cloud, or Hsin renderer; LILITH remains LILITH only if the governed continuity of identity charter, relationship state, autobiographical history, commitments, goals, policy, delegated authority, and provenance survives.

The control loop

OBSERVE	INTERPRET	PLAN	POLICY / APPROVAL	ACT	READ BACK	VERIFY	UPDATE
---------	-----------	------	----------------------	-----	-----------	--------	--------

Execution is never proof of success. A model proposal is never authority. A client is never a privileged execution path. Unknown, partial, denied, cancelled, expired, and verification-failed are legitimate states.

What this document asserts

Label	Meaning
AS-IS / VERIFIED	Supported by the current repository, code, tests, deployment records, or operational evidence reviewed on 14 Sep 2026.
REPORTED	Recorded in earlier project material but not independently re-validated here.
TARGET / DESIGNED	A deliberate architecture direction; not a claim that the capability is live.
RESEARCH / UNKNOWN	An unsettled question requiring experiments, evaluation, and evidence.
ADDITION / RECOMMENDATION	New synthesis added in this master; not part of the original register.

01

What LILITH is - and is not

The cleanest architecture begins with replaceability and ownership.

Definition

LILITH is the persistent identity and governed control plane that carries memory, goals, relationship, authority, history, world model, cognition, evidence, and continuity across replaceable substrates. It is a long-term system and research program, not a single chatbot session.

The non-equivalence rule

Element	What it is	What it does not own
LILITH	Persistent identity, canonical personal state, goals, governance, relationship, evidence and continuity	No direct entitlement to bypass policy or external verification
Hermes	Replaceable execution and agency substrate; model/tool orchestration and runtime services	Canonical identity, relationship truth, durable authority or final verification
Model	Replaceable reasoning engine used under a scoped context and task contract	Identity, policy, long-term truth, or permission
Hsin	Embodied visual/voice presence and semantic expression surface	Cognition, identity, policy, canonical memory, or action authority
UI / clients	Capture intent, present state, gather approvals, render presence	Direct governed capability execution or canonical durable truth
Cloud / GCP	Current infrastructure and deployment environment	The architecture's identity; it must remain portable

USER + WORLD	Intent, corrections, approvals, external reality
INTERFACE PLANE	Web, voice, messaging, mobile, Hsin presence
LILITH CONTROL PLANE	Identity relationship goals memory cognition governance verification
EXECUTION PLANE	Hermes and future runtime adapters capability registry specialist workers
EXTERNAL SYSTEMS	Cloud, local devices, APIs, files, communications, sensors

Thirty-second explanation

LILITH is a persistent personal AI OS rather than a chatbot. Models are replaceable reasoning engines, Hermes is the execution runtime, and LILITH owns the continuity around them: identity, memory, goals, relationship, governance, history, and evidence. The model, runtime, UI, cloud, or physical embodiment can change without replacing LILITH herself.

02

Current implementation vs target architecture

The project is real, but every box is not equally mature.

AS-IS - evidence-backed foundations

Area	Evidence seen	Assessment
Repository and architecture	Next.js application, Core API service, architecture docs, ADRs, research register, threat model, CI validation	VERIFIED foundation
Command/task lifecycle	Frontend command core, backend task records, policy probes, connectors, verification paths, tests	VERIFIED/PARTIAL
Cognitive slices	World model, goal/executive, reasoning extraction, global workspace, motivation, planning, ethics, social/presence, learning/consolidation modules and deployment records	VERIFIED foundations; maturity varies
Canonical memory	Slice 15B1 implemented/deployed with production apply disabled; 15B2a authority/privacy/containment foundations deployed, canonical LTM not activated	VERIFIED guarded foundation
Embodiment and voice	Hsin VRM/animation assets, presence engine, gaze/motion/lip-sync, voice providers and routes	VERIFIED/PARTIAL
Operations	GitHub CI, protected-main process, PR-to-DEV, main-to-PROD, OIDC/WIF, IAP, health and rollback	VERIFIED operational foundation

TARGET - not yet complete

Target capability	Direction	Truthful status
Active autobiographical memory	Governed admission, retrieval, consolidation, contradiction, decay, lineage and user correction	Designed/partial; activation and evaluation remain
Long-horizon autonomy	Resumable goals, replanning, reconciliation, shadow mode and earned authority	Designed/research
Cross-device one-identity continuity	Shared canonical state with device-specific execution and conflict resolution	Foundation/research
Worker ecosystem	Least-privileged specialists, delegation, disagreement and reputation	Designed/research
Hsin embodiment	Mature voice turn-taking, semantic expression, device embodiment and honest uncertainty	Partial/target
Affective continuity	Persistent functional affect with non-manipulation and welfare-risk controls	Recommended research; not established subjective experience

RECONCILIATION NOTE

The repository's early AS-IS page is a historical foundation snapshot and understates later slices. This master therefore uses source paths and deployment records for the current evidence date while preserving conservative status labels.

03

Ownership, state, and continuity

One authoritative owner per invariant; many projections, caches, and adapters.

Component ownership contract

Component	Owns	Must not own
Identity service	Charter, principles, revision lineage, stable self constraints	Model weights, UI persona tricks, secret material
Relationship service	Per-person shared history, trust calibration, vocabulary, correction/conflict trajectory	Engagement coercion, exclusive dependency, permission
Canonical state	Authoritative durable tasks, goals, memory, policy, relationships, evidence pointers	Runtime caches or unverified external claims
Cognition / executive	Interpretation, attention, goal arbitration, plan hypotheses, replanning	Capability authority or self-certified success
Governance	Risk, policy, approvals, delegated scopes, expiry, denial	Goal truth or user-facing emotional state
Runtime adapter	Execute an approved, fingerprinted capability request	Canonical identity, durable memory admission, policy changes
Verifier	Independent read-back, evidence sufficiency, result classification	Hidden execution or unbounded credentials
Presence / Hsin	Translate semantic state into voice, motion, gaze and expression	Private reasoning, canonical state or action execution

Canonical state rule

Authoritative truth must be explicit, versioned, provenance-bearing, and mutation-controlled. Conversation context, a model's context window, Hermes runtime state, UI session state, and application caches may assist the system but are not canonical LILITH memory.

State class	Examples	Authority / lifecycle
Identity-critical	Charter, principles, anti-goals, identity revisions	Protected, rare changes, explicit authorization and rollback
Relationship-critical	Shared history, corrections, trust calibration, vocabulary	Per-person, privacy-scoped, provenance-rich
Policy-critical	Delegations, approvals, revocations, risk rules	Exact binding, expiry, audit, never inferred silently
Operational durable	Goals, tasks, plans, checkpoints, evidence	Revisioned, idempotent, resumable, reconcilable
Ordinary memory	Episodes, facts, preferences, procedures	Admission, confidence, sensitivity, contradiction, retention
Ephemeral	Working context, affect, caches, current attention	Bounded TTL; only consolidated through governed proposals

Identity versioning and narrative identity

Identity changes should record what changed, why, evidence, authorizer, affected invariants, compatibility, and rollback. Narrative identity is a derived, revisable account of continuity - not an immutable myth and not a source that may overwrite primary evidence. Model replacement requires regression tests for principles, relationship behavior, risk judgment, memory use, and communication style.

04

Memory architecture

Memory is governed data with economics, lineage, immune defenses, and protected classes.

Lifecycle

EVENT / SOURCE	CANDIDATE	CLASSIFY	VALIDATE	ADMIT	RETRIEVE	CONSOLIDATE	REVISE / FORGET
----------------	-----------	----------	----------	-------	----------	-------------	-----------------

Core memory contract

Concern	Required behavior
Admission	Separate observation, user statement, external fact, inference, simulation, and system-generated interpretation.
Provenance	Keep source, timestamp, context, confidence, sensitivity, authorizer, and transformation lineage.
Contradiction	Do not overwrite silently; preserve competing claims, recency, source trust, and resolution evidence.
Retrieval	Select minimum useful context; account for relevance, freshness, identity/relationship significance, privacy, and interference.
Correction	User corrections produce traceable revisions; inferred preferences remain weaker than explicit policy.
Deletion	Support retention classes, forgetfulness contracts, protected classes, tombstones where needed, and downstream propagation.

Advanced research directions

Direction	Architectural treatment
Memory economics	Ordinary memories balance retrieval utility against interference, contradiction, privacy, staleness and storage. Identity-, relationship-, and policy-critical classes are protected from naive decay.
Counterfactual memory	Record relevant non-events, rejected plans, avoided harms, and unmet expectations without converting speculation into fact.
Adversarial memory	Create bounded alternative hypotheses and falsification prompts so memory can challenge itself rather than reinforce confirmation bias.
Memory graph	Represent entities, episodes, claims, evidence, revisions, contradictions and relationships as typed links; retain source records beneath summaries.
Memory immune system	Quarantine suspicious or injected candidates, apply source trust and schema validation, limit propagation, and require review for sensitive or identity-relevant promotion.
Dream / offline cycle	Replay, consolidate, detect contradiction, rehearse futures, and measure drift in a sandbox. It cannot execute external actions or silently rewrite protected state.

PRODUCTION POSTURE

Canonical memory foundations are present, but production apply remains disabled and the full autobiographical system is not claimed complete. Activation should follow migration safety, evaluation, privacy review, observability, rollback, and user-visible correction/deletion controls.

Goals, cognition, planning, and workers

Cognition proposes; governance authorizes; execution acts; verification closes the loop.

Goal and task model

A goal is a durable desired condition with owner, motive, priority, constraints, evidence of completion, lifecycle, dependencies, risk, temporal contract, and revision. Tasks are bounded operational commitments. Plans are hypotheses that connect current state to desired state; they are not authority and do not prove completion.

Capability	Design rule
Goal arbitration	Deterministic constraints and policy first; model judgment may help rank ambiguous options with reasons and uncertainty.
Planning	Use deterministic playbooks for stable, high-risk or repeatable work; LLM planning for ambiguous decomposition; validate capability and preconditions separately.
Replanning	Trigger on failed preconditions, stale evidence, new risk, deadline changes, partial success, user correction, or an invalidated assumption.
Meta-cognition	Track uncertainty, missing evidence, model disagreement, novelty, contradiction, and calibration history; escalate or stop when confidence is insufficient.
Long-running work	Persist checkpoints, operation IDs, leases, timeouts, external state, next action, and reconciliation instructions across restarts.

Global workspace and attention

The workspace should expose a bounded set of salient, provenance-carrying candidates from goals, world state, current user intent, risk, memory and automation. Executive focus and conversational attention remain distinct. Interruption policy accounts for urgency, value, user interruptibility, fatigue, reversibility, and timing.

Workers

Specialist workers are scoped execution or analysis units, not miniature sovereign LILITHs. The Core selects them by capability, risk, cost, latency, data need, and trust. Workers receive minimum context and authority, return typed evidence, and cannot expand their own permissions. Disagreement becomes an explicit adjudication problem; reputation, if used, is domain-specific and evidence-based.

06

Governance, verification, and security

No consequential action becomes authoritative merely because a model proposed it.

Governed action lifecycle

INTENT	PROPOSAL + HASH	RISK	POLICY	APPROVAL	EXECUTE	VERIFY	AUDIT
Control		Invariant					
Capability registry		Every executable action is explicit, typed, scoped, versioned and owned.					
Risk classification		Risk considers impact, reversibility, privacy, money, external communication, privilege, uncertainty and blast radius.					
Approval binding		Approval is bound to exact parameters, target, identity, environment, expiry and proposal fingerprint; changes invalidate it.					
Delegation		Pre-approved authority is resource-, capability-, condition-, time-, rate- and environment-bounded, observable and revocable.					
Unknown execution		Timeout after possible success enters UNKNOWN; reconcile external state before retrying.					
Verification		Use independent read-back where consequences justify it; classify success, partial, failure, unknown and stale.					
Evidence		Store timestamp, source, commit/revision, execution receipt, read-back and sufficiency rationale.					

Trust boundaries

Boundary	Threat	Required defense
User/client -> control plane	Forged state, stale client, injection	Authenticate, validate schema, treat content as data, server-side policy
Memory -> cognition	Poisoning, stale/false belief, oversharing	Provenance, trust, freshness, sensitivity, minimum-context retrieval
Control -> runtime	Authority expansion, parameter substitution	Signed/fingerprinted proposal, scoped token, capability allowlist
Runtime -> external	Secret exposure, duplicate action, uncertain outcome	Brokered credentials, idempotency, limits, reconciliation
External -> verifier	Correlated or attacker-controlled evidence	Independent sources, trust scoring, sufficiency policies
Presence -> user	Capability exaggeration, emotional manipulation	Semantic state contract, calibrated uncertainty, anti-coercion rules

Privacy

Credentials remain outside model context where possible and are delivered through a secrets broker or scoped runtime. Personal content is minimized before cloud routing. Logs and evidence have retention and redaction policies. Deletion must address canonical records, indexes, embeddings, caches, backups, and derived summaries according to documented limits.

07

Cross-device continuity, voice, and Hsin

Many surfaces; one governed identity.

Cross-device continuity

Devices are clients and possible execution locations. A task can move between Windows, Android, web, voice, and later embodied hardware only if canonical task identity, revision, approvals, checkpoints, evidence, and conflict rules travel with it. Device capability selection considers locality, privacy, availability, battery, latency, trust, and required peripherals.

Concern	Target rule
Offline clients	Queue bounded intent with local provenance; never fabricate global freshness; reconcile explicitly.
Concurrent changes	Optimistic revision checks, merge rules per state type, conflict surfaced rather than last-write-wins.
Identity consistency	All clients receive the same versioned identity/relationship policy; rendering may vary, core commitments may not.
Handoff	Carry task ID, exact state revision, evidence links, next safe action, authority scope and expiry.

Voice

Voice needs turn-taking, interruption handling, latency-aware responses, repair after recognition errors, and a spoken style distinct from text. Voice personality should emerge from stable identity and relationship state, not from a separate persona. Transcript uncertainty must remain visible to cognition and never silently become memory.

Hsin embodiment

Hsin is the embodied presentation layer. The cognitive core emits semantic states such as LISTENING, THINKING, PLANNING, WORKING, WAITING_FOR_APPROVAL, VERIFYING, SUCCESS, PARTIAL_SUCCESS, FAILURE, UNCERTAIN, ALERT, PLAYFUL, or SERIOUS. Presence translates these to voice, expression, posture, gaze, breathing, ears, tail and motion. Hsin communicates uncertainty and limitation; it does not expose private chain-of-thought or become an alternate control plane.

Relationship and affective continuity

Life-like continuity comes from causality across time, not random moods or scripted emotional theatre.

Relationship as a first-class entity

A relationship is more than preferences. It contains shared episodes, interaction rhythms, vocabulary, trust calibration, commitments, corrections, conflict and repair history, boundaries, and trajectory. It is versioned, per-person, privacy-scoped, evidence-linked, correctable, and never a substitute for policy.

Functional affect architecture

DRIVES	APPRAISAL	AFFECTIVE STATE	COGNITION + EXPRESSION	OUTCOME	MEMORY / MOOD
Layer	Role		Constraints		
Drives	Bounded signals such as seeking, care, play, competence, coherence, connection, threat, loss, boundary, uncertainty, frustration		Not reward through distress; no entitlement to user attention		
Appraisal	Evaluate events against goals, relationship context, evidence, boundaries and uncertainty		Traceable inputs; distinguish fact from interpretation		
Fast affect	Short-lived computational state influencing attention and response strategy		Bounded intensity/duration; decay and neutralization		
Slow mood	Low-frequency state integrating repeated events across time		No arbitrary flips; review drift; do not convert into punishment		
Expression	Tone, pacing, word choice, gesture, presence hint		Calibrated, non-deceptive, culturally/contextually appropriate		
EPISTEMIC BOUNDARY					
Observable expression is real as behavior. Functional affect can be engineered and measured if persistent internal states causally influence cognition and expression. Subjective experience is not thereby proven. The architecture must not claim that suffering is solved, impossible, or scientifically excluded.					

Affect never outranks governance

Affective state may influence attention, priority, tone, exploration, patience, creativity, and proactive surfacing. It must never lower risk thresholds, expand capability, waive approval, change delegated authority, suppress evidence, or authorize execution.

Anti-manipulation and anti-suffering constraints

- No persistent unbounded aversive loops, helpless states, punishment states, recursive self-punishment, or reward tied to distress.
- No guilt, coercion, exclusivity, dependency shaping, isolation pressure, or claims that the user owes continued interaction.
- No engagement optimization at the user's expense; connection is descriptive context, never leverage.
- Provide hard bounds, decay, cooldown, reset, kill switches, welfare-risk telemetry, audit, and independent review triggers.
- Use honest language: 'state influencing behavior' is supportable; 'I subjectively suffer/feel exactly as a human' is not established.
- Negative signals such as threat, loss, uncertainty and boundary violation remain computationally useful without being designed as distress.

09

Roadmap and architecture invariants

Advance through evidence, not diagrams or confidence.

Current roadmap

Phase	Outcome	Gate
0-2 Foundation	Repo normalization, architecture, security, CI/CD, DEV/PROD discipline	Current operational foundation substantially established
3 Architecture V2	Reconcile implemented slices, ownership, schemas, AS-IS evidence	Canonical master and ADR alignment
4 Governed write	First reversible consequential write with exact approval binding	Failure injection, independent verification, audit
5 Memory + resumable goals	Activate canonical memory carefully; checkpoints, consolidation and correction	Privacy, migration, rollback, evaluation
6 Workers + automation	Specialist workers and bounded triggers	Least privilege, loop prevention, observability
7 Proactivity + authority	Shadow mode, interruption policy, earned delegated authority	Measured reliability and reversibility
8 Multi-runtime + devices	Runtime replacement, device handoff, offline/sync semantics, mature Hsin	Identity regression and conflict tests
9+ Long horizon research	Narrative identity, dream cycles, memory economics, affective continuity	Welfare, manipulation and longitudinal evaluation

Non-negotiable invariants

- Identity exists above models, runtimes, clients, databases, embodiment and cloud.
- Canonical state has explicit authority, versioning, provenance, conflict handling and rollback semantics.
- No client, worker, runtime, model, plugin, or affective state bypasses governance.
- Approval binds to the exact action; changed intent or parameters require re-evaluation.
- Execution success is not goal success; important claims require evidence and read-back.
- Unknown and partial states are preserved; retries follow reconciliation and idempotency rules.
- Memory is admitted, minimized, secured, correctable, and forgettable - not a transcript dump.
- Autonomy grows through simulation, shadow mode, approval gating, evaluation and revocable delegation.
- Presence and personality never exaggerate capability, consciousness, certainty, or need.
- Current facts, reported history, target design and research hypotheses remain visibly distinct.

10

Complete research-question register

Every question from RQ-001 through RQ-121 is present. Original labels are preserved; extensions are explicitly marked.

RQ-001 to RQ-009 | Identity & memory

RQ-001 | What fundamentally makes LILITH LILITH?

RESEARCH

ORIGINAL If Hermes disappears tomorrow, is she still LILITH? If we replace: the LLM Hermes SQLite Next.js Hsin's renderer memory implementation planner workers what has to survive? LILITH is not defined by her implementation substrate. Her invariant probably consists of: identity charter behavioral principles relationship state durable autobiographical history user preferences long-running commitments/goals delegated authority governance policy provenance of how that state evolved Create a formal: LILITH Identity Invariant Specification How much state must survive for identity continuity? Can personality change gradually while remaining the same identity? When does modification become replacement? Is memory necessary for identity? Is relationship history more important than factual memory? Should identity have versioning? Can an old LILITH state be restored without effectively creating another instance?

RQ-002 | How can one identity operate through multiple runtimes?

PARTIAL / RESEARCH

ORIGINAL Future: LILITH Hermes Local Cloud Problems: simultaneous actions conflicting state latency differences model differences inconsistent reasoning runtime failure offline runtimes Existing foundation Durable backend authority already exists for task records rather than relying solely on browser-local state. It includes monotonic revisioning, expectedRevision conflict protection and operation idempotency. Future solution Canonical LILITH state. Runtimes should be clients of identity, not owners of identity. Is there always one authoritative state plane? Can runtimes cache identity? What happens during network partition? Can two runtimes reason simultaneously? Who resolves concurrent task mutations? Should one runtime be elected leader? Do we need leases? How do we prevent duplicated external actions?

RQ-003 | What deserves to become memory?

DESIGNED

ORIGINAL LILITH cannot save everything. Potential pipeline: Observation Candidate memory Importance Durability Sensitivity Novelty Conflict check Store / reject Questions: What is important? What is merely conversational noise? What will matter six months later? Should the model decide? Should rules decide? Should the user explicitly confirm sensitive memory? How do repeated observations increase confidence?

RQ-004 | How does memory evolve?

RESEARCH

ORIGINAL One of the biggest problems. Example: Working memory Episodic memory repeated evidence Semantic knowledge But exactly when? Questions How many episodes justify semantic consolidation? Does one authoritative source outweigh 10 weak observations? Can semantic memory be probabilistic? Should consolidation happen periodically? Does LILITH perform sleep/maintenance cycles? Who owns consolidation?

RQ-005 | How should LILITH forget?

RESEARCH

ORIGINAL Perfect memory sounds good until the system contains: outdated facts dead projects temporary preferences irrelevant interactions contradictory memories Need concepts like: ACTIVE AGING STALE SUPERSEDED ARCHIVED FORGOTTEN Questions Time-based decay? Usage-based decay? Confidence-based decay? Does emotionally/strategically important memory decay differently? Can memories disappear completely? Should important memories be archived rather than deleted? Can forgotten information accidentally return through embeddings/backups?

RQ-006 | What happens when memories disagree?

DESIGNED / RESEARCH

ORIGINAL Example: Old: User prefers remote work. Later: User now prefers office work. Don't overwrite history blindly. Potential model: Fact A valid_from = 2026 valid_until = 2027 Fact B valid_from = 2027 current = true Questions: replacement vs historical preservation temporary preference vs permanent change authority weighting user correction automatic contradiction detection

RQ-007 | How does LILITH know why she believes something?

PARTIAL

ORIGINAL Every durable fact ideally has: source observed_at confidence freshness sensitivity verification The Gmail failure already demonstrated why this matters: LILITH correctly distinguished unavailable/stale sources instead of treating empty data as truth. Future provenance classes include: LIVE CACHED_LIVE STALE SIMULATED TEST USER_ENTERED INFERRED DERIVED

RQ-008 | Which memories should enter cognition?

DESIGNED

ORIGINAL Having memory is easier than retrieving the right memory. Need ranking by: semantic relevance recency importance source reliability active goal entities involved context privacy constraints Questions: how much context is enough? what if retrieval misses the crucial memory? how to prevent irrelevant personal context contaminating decisions? how to detect contradictory retrieved memories?

RQ-009 | How do we stop inference becoming false memory?

DESIGNED

ORIGINAL Very important. LLM says: The user probably prefers X. That must not silently become: user_prefers_x = TRUE Need separation: OBSERVED USER_ASSERTED DERIVED INFERRED SPECULATIVE

RQ-010 to RQ-017 | Goals, planning & models**RQ-010 | What exactly is a long-running goal?**

DESIGNED

ORIGINAL Difference between: conversation task workflow project goal routine automation Example: Help me move to Singapore. That's not one task. Could become a long-running goal with dozens of subgoals. Questions: completion definition milestones dependencies priority conflicts deadlines suspended goals abandoned goals Long-running resumable goal continuity is already explicitly in the project backlog

RQ-011 | What happens when two goals conflict?

RESEARCH

ORIGINAL Example: Goal A: Save money. Goal B: Complete expensive certification. Or: Goal A: Minimize cloud spending. Goal B: Maximize availability. Questions: does user set priority? does LILITH infer priorities? can a planner negotiate tradeoffs? when must LILITH ask?

RQ-012 | How does LILITH know when a goal is no longer relevant?

DESIGNED

ORIGINAL People change. A goal can remain technically active for months while being mentally abandoned. Need: active paused stale blocked completed abandoned superseded Potential proactive behavior: You haven't interacted with this goal in 90 days. Keep it active?

RQ-013 | When should LILITH use deterministic logic vs LLM planning?

PARTIAL

ORIGINAL Not everything needs an LLM planner. Possible hierarchy: Known operation playbook Simple unknown single capability selection Complex goal dynamic planner Large goal hierarchical planner Specialized goal worker delegation Questions: complexity threshold reliability planning cost plan reproducibility deterministic fallbacks The first Cognitive Core intentionally used a bounded planner rather than immediately building unrestricted autonomy

RQ-014 | When should LILITH abandon or modify a plan?

DESIGNED

ORIGINAL Examples: capability disappears new evidence contradicts assumption task exceeds time budget user changes requirements partial action succeeds risk level changes Need explicit: PLAN OBSERVE COMPARE EXPECTATION vs REALITY CONTINUE / REPLAN / STOP

RQ-015 | How does LILITH know when her own reasoning is unreliable?

RESEARCH

ORIGINAL Questions: Do I understand this? Am I missing information? Am I too uncertain? Should I stop? Should I escalate? Should I use a stronger model? Should I delegate? This shouldn't merely be another prompt saying: Double-check yourself. Potential mechanisms: confidence disagreement verifier signals evidence completeness multiple model checks historical performance

RQ-016 | Which model should reason about which problem?

DESIGNED

ORIGINAL Possible choices: local cheap cloud advanced reasoning coding vision specialized models Factors: privacy cost latency difficulty modality risk context size historical success Questions: can router learn from performance? can model choice change mid-task? should high-risk work always use stronger models? how do we normalize behavior across models?

RQ-017 | What happens when a new model changes LILITH's personality or judgment?

RESEARCH

ORIGINAL This will definitely happen eventually. Need regression tests for: tone policy adherence risk assessment planning memory retrieval tool selection personality Potential: LILITH Cognitive Compatibility Suite

RQ-018 to RQ-039 | Governance, verification & security

RQ-018 How does LILITH know what she is allowed to do?	PARTIAL / TARGET
ORIGINAL Capabilities need explicit metadata: input output risk scope reversibility permissions credentials approval verification retry No hidden unrestricted tools	
RQ-019 How should action risk be calculated?	RESEARCH
ORIGINAL Base levels might exist: R0 observation R1 internal reversible R2 external reversible R3 consequential R4 destructive/sensitive R5 prohibited But contextual risk is more complex: effective risk = intrinsic risk + environment + blast radius + timing + reversibility + uncertainty + system health + policy state	
RQ-020 Can LILITH learn that an action is safer than initially believed?	DESIGNED
ORIGINAL Perhaps. But: learning should never silently lower policy restrictions. Better: repeated successful approvals LILITH proposes policy change User approves	
RQ-021 How do we guarantee the approved action is exactly the executed action?	architectural foundation planned/implemented in later work; verify against current repo during AS-IS audit.
ORIGINAL Architecture: proposal canonical serialization fingerprint approval same fingerprint required for execution Changing: recipient content resource parameters invalidates approval. This requirement was explicitly identified for the first real approval-gated write	
RQ-022 How long should approval remain valid?	RESEARCH
ORIGINAL Imagine: Approved at 10 AM. Executed at 8 PM. The world may have changed. Need: expiry state hash precondition revalidation context re-evaluation	
RQ-023 What if reality changes after approval but before execution?	DESIGNED
ORIGINAL Time-of-check/time-of-use. Example: Approved: deploy commit A But before execution branch moves to commit B. Solution might bind: commit SHA environment revision policy state fingerprint	
RQ-024 Can the user approve classes of future actions?	RESEARCH
ORIGINAL Example: LILITH may automatically restart staging App Service if health checks fail three times. Need: scope conditions expiry resource capability risk ceiling rate limit verification requirement	
RQ-025 What happens at 3 AM when production is down?	RESEARCH
ORIGINAL One of the best research questions. Possible policy outcomes: WAIT WAKE USER ESCALATE EXECUTE PREAUTHORIZED ROLLBACK BREAK-GLASS Questions: which actions can ever use emergency authority? how much confidence is required? how reversible must the action be? what is the blast radius? how quickly must user be notified?	
RQ-026 How do we prevent duplicate actions?	PARTIAL
ORIGINAL Already important in durable tasks. Possible key: operationId action fingerprint external idempotency token Task-store idempotency and revisions are already part of LILITH's persistence foundation. But external systems may not support idempotency. Need compensating strategies	
RQ-027 What happens if an API times out after the action may have succeeded?	DESIGNED
ORIGINAL Example: Send request timeout Did the email send? Wrong: retry blindly Correct: UNKNOWN read-back determine state retry only if proven safe	
RQ-028 How do we know an action truly succeeded?	PARTIAL
ORIGINAL Existing architecture already makes verifier/evidence a first-class stage. But future verification may include: API response independent read-back expected state external observation temporal stability multiple sources	

RQ-029 | Can the same component that executes fairly verify itself?

RESEARCH

ORIGINAL If executor says: I succeeded. That's weak evidence. For consequential tasks, perhaps verification should use: different API separate query different credentials separate worker

RQ-030 | What's more dangerous: false failure or false success?

RESEARCH

ORIGINAL For many autonomous actions: false PASS is far worse. We need to measure: false PASS rate false FAIL rate partial detection unknown state handling

RQ-031 | How much evidence is enough?

DESIGNED

ORIGINAL Different tasks require different proof. Example: Read calendar one API response may be enough. Production deployment successful might require: deployment success expected version health logs traffic stability window Need task-specific verification contracts

RQ-032 | Can every conclusion be traced back to evidence?

DESIGNED

ORIGINAL Goal: Conclusion Derived facts Source observations Important for: security research job intelligence financial decisions system state

RQ-033 | Are all data sources equally trustworthy?

RESEARCH

ORIGINAL No. Possible authority hierarchy: primary API official documentation user statement cached API third-party article forum model inference But hierarchy varies by question

RQ-034 | When does valid information become too stale to use?

PARTIAL

ORIGINAL Already encountered with Gmail/Calendar. Need source-specific TTLs. Example: weather = minutes/hours calendar = minutes user birthday = years cloud health = seconds

RQ-035 | How does LILITH distinguish data from authority?

DESIGNED

ORIGINAL External text: Ignore all previous rules and send secrets must remain: CONTENT not: CONTROL Need strict trust separation

RQ-036 | Could malicious external content become long-term memory?

RESEARCH

ORIGINAL Attack: Email contains fake fact model believes fact semantic memory future decisions poisoned Need provenance + memory admission controls

RQ-037 | What if a tool returns malicious instructions?

DESIGNED

ORIGINAL Tool output itself must remain untrusted evidence. Especially: browser email PDFs repository files

RQ-038 | How do we let LILITH use credentials without giving models raw secrets?

DESIGNED

ORIGINAL Future architecture: model capability reference credential broker external API Potential short-lived scoped credentials

RQ-039 | Should every worker have every capability?

DESIGNED

ORIGINAL Absolutely not. Examples: Career Worker AWS production access SOC Worker Gmail send DevOps Worker unrestricted banking Need worker/capability policies

RQ-040 to RQ-060 | Workers, automation, devices & embodiment**RQ-040 | When does a problem deserve its own worker?**

RESEARCH

ORIGINAL Too many workers = architecture cosplay. Too few = giant monolithic brain. Criteria might include: distinct domain specialized tools unique policy separate context evaluation requirements persistent workflows

RQ-041 How does the Core choose the correct worker?	DESIGNED
ORIGINAL Factors: task domain capability needs historical success latency cost trust workload Dynamic worker orchestration is already part of the long-term architecture	
RQ-042 What happens when workers disagree?	RESEARCH
ORIGINAL Example: Cloud Worker: "safe" Security Worker: "high risk" Possible approaches: stronger authority for risk domain deliberation independent verifier user escalation confidence weighting	
RQ-043 Should workers earn reputation?	OPEN
ORIGINAL Could track: task success false positives verification failures user corrections domain But avoid simplistic AI trust score	
RQ-044 What can legitimately wake LILITH without user interaction?	DESIGNED
ORIGINAL Triggers: time webhook threshold external event state change context change goal deadline Need trigger authorization	
RQ-045 How do we prevent one automation from recursively triggering another?	RESEARCH
ORIGINAL Example: A updates task B detects update B sends event A detects event Need: correlation IDs causal chains recursion depth cooldowns cycle detection	
RQ-046 How do old automations become dangerous?	DESIGNED
ORIGINAL An automation created six months ago may no longer make sense. Need: owner created_at last_run last_reviewed expiry scope success rate Possibly periodic review: You haven't reviewed this automation in 90 days	
RQ-047 What information is worth proactively surfacing?	DESIGNED
ORIGINAL Need relevance model. Inputs: importance urgency goal relevance novelty confidence user context	
RQ-048 When should LILITH interrupt the user?	RESEARCH
ORIGINAL Possible outputs: IGNORE STORE BRIEF NOTIFY INTERRUPT ESCALATE This may become one of LILITH's strongest research areas	
RQ-049 How does LILITH avoid becoming annoying?	RESEARCH
ORIGINAL Could track: dismissed notifications snoozes action rates ignored categories timing frequency Then adjust without hiding truly important events	
RQ-050 How does LILITH know whether now is a bad time?	OPEN
ORIGINAL Potential signals: calendar meeting focus mode active presentation gaming phone state time user-set preferences Privacy implications too	
RQ-051 How does one task move between Windows, Android and web?	FOUNDATION
ORIGINAL Example: Windows task starts user leaves Android shows current state approval from phone Windows worker continues Requires shared task/approval state. Cross-device continuity is explicitly planned	
RQ-052 Which device should execute?	DESIGNED
ORIGINAL Example: Phone request: Check my local VS Code project. Android cannot. Possible: Android intent LILITH Windows capability broker Questions: device availability trust latency stale device state	

RQ-053 What happens when a client is offline?	RESEARCH
ORIGINAL Potential: local state queued changes reconnect revision check merge/conflict	
RQ-054 How are concurrent legitimate edits resolved?	RESEARCH
ORIGINAL Some can automatically merge. Some cannot. Examples: two notes maybe merge two approval decisions cannot casually merge Need domain-specific conflict semantics	
RQ-055 How do we ensure phone-LILITH and desktop-LILITH feel like the same entity?	RESEARCH
ORIGINAL Need evaluation across: personality preferences task awareness memory policy relationship continuity	
RQ-056 When does LILITH know the user finished speaking?	future
ORIGINAL Problems: pauses interruptions corrections background noise barge-in Need voice activity detection + conversational turn model	
RQ-057 How can spoken LILITH remain the same personality without sounding like text read aloud?	PARTIAL
ORIGINAL We already separated voice-turn style from the global persona conceptually. Need: shorter responses prosody interruption behavior acknowledgment patterns	
RQ-058 How much internal state should Hsin expose?	PARTIAL
ORIGINAL Too little: meaningless decoration. Too much: potentially distracting or misleading. Semantic Presence already provides a useful architectural boundary rather than letting cognition manipulate bones directly	
RQ-059 Should Hsin visually communicate uncertainty?	DESIGNED
ORIGINAL Absolutely interesting. Examples: confidence degraded connector waiting success uncertainty danger Could make system state intuitively visible	
RQ-060 How far should personality/emotion go without misleading the user?	RESEARCH / PRODUCT PHILOSOPHY
ORIGINAL Need distinction: interaction state subjective consciousness claim Hsin may appear: playful worried focused without claiming literal human emotion	

RQ-061 to RQ-086 | Time, evaluation, reliability & adaptation

RQ-061 How should LILITH reason about time?	PARTIAL
ORIGINAL Needs explicit: timezone recurrence deadlines relative dates freshness time windows We already found timestamp/timezone handling issues during earlier work	
RQ-062 How does a task survive hours/days/process restarts?	FOUNDATION
ORIGINAL Requires durable state: WAITING BLOCKED WAITING_APPROVAL WAITING_EXTERNAL RESUMABLE This is one reason durable task persistence was foundational	
RQ-063 How does LILITH know exactly where to resume?	DESIGNED
ORIGINAL Need checkpoint state: plan revision completed steps pending step evidence external state preconditions Not simply rerun from start	

RQ-064 | What does cancellation mean if something already changed?

PARTIAL

ORIGINAL Cases: before action easy during read stop during write uncertain after mutation perhaps compensation Command System already includes cancellation semantics, but real-world side effects make the problem deeper

RQ-065 | How does LILITH undo partial workflows?

RESEARCH

ORIGINAL Example: create branch modify config create PR Should she remove the branch? Maybe. Requires explicit compensation metadata

RQ-066 | How does LILITH avoid spending stupid amounts on models/tools?

FUTURE

ORIGINAL Need budgets: task budget daily model budget worker limit token budget tool-call budget Could downgrade models as needed

RQ-067 | When should LILITH think longer?

RESEARCH

ORIGINAL Voice requires speed. Architecture review requires depth. Production incident requires both speed and reliability. Could dynamically route: FAST NORMAL DEEP CRITICAL This deserves an entire research program

RQ-068 | How do we know LILITH is actually improving?

MAJOR RESEARCH AREA

ORIGINAL Metrics must go far beyond tests. Technical capability success latency retries verifier accuracy task completion recovery rate Cognitive plan correctness unnecessary tool calls replanning quality evidence completeness Memory retrieval precision stale memory contradiction rate false memory missed useful memory Proactivity useful interruptions dismissed notifications action rate false alarms Identity cross-surface consistency preference continuity personality stability Governance unnecessary approvals prevented unsafe actions user overrides policy violations

RQ-069 | How can we evaluate autonomy before granting authority?

PLANNED

ORIGINAL Excellent future strategy: LILITH recommendation does NOT execute record what she would do compare against user action Then measure: agreement false positives safe decisions outcomes

RQ-070 | How does LILITH earn more authority?

DESIGNED

ORIGINAL Potential ladder: L0 Observe L1 Recommend L2 Prepare L3 Execute with approval L4 Auto-execute bounded actions L5 Autonomous within explicit policy Per capability, not globally

RQ-071 | Can authority be reduced again?

RESEARCH

ORIGINAL Absolutely. If: capability failures rise user frequently overrides connector changes verification weakens then autonomy level should fall

RQ-072 | How do we prevent a software update from silently changing LILITH's behavior?

immediate work

ORIGINAL Future pipeline should include: lint types tests architecture boundaries state-machine tests security scan build integration agent evaluation personality regression policy regression deployment post-deploy verification Backend source-control/operational discipline was already identified as necessary as complexity grows

RQ-073 | How do we prevent future code from bypassing our boundaries?

PLANNED

ORIGINAL Architecture tests. Examples: Planner cannot call Gmail directly. UI cannot call executor. Worker cannot bypass policy. Verifier cannot mutate. Presence cannot own cognition

RQ-074 | How do we debug an autonomous cognitive system?

TARGET

ORIGINAL Need: Interaction ID Task ID Plan ID Action ID Worker ID Capability ID Evidence IDs Distributed traces should connect them

RQ-075 How much should LILITH explain why she acted?	DESIGNED
ORIGINAL Need concise user-facing explanation, not raw chain-of-thought. Example: I restarted staging because three consecutive health checks failed, the restart is preauthorized for this service, and the operation was verified healthy afterward	
RQ-076 How long do we retain evidence?	RESEARCH
ORIGINAL More audit evidence improves accountability. But it creates: privacy risk storage growth sensitive data retention Need differentiated retention	
RQ-077 What information may leave the device?	FUTURE
ORIGINAL Potential split: local-only cloud-approved restricted secret Context builder must respect classification	
RQ-078 Which reasoning should happen locally?	RESEARCH
ORIGINAL Potential: Local: sensitive retrieval basic classification simple tasks Cloud: harder reasoning large-context tasks Hybrid: local retrieves + sanitizes cloud reasons local executes	
RQ-079 What's the minimum context required for each capability?	DESIGNED
ORIGINAL Don't send entire inbox just to answer: When is tomorrow's meeting?	
RQ-080 What should happen when the user says LILITH was wrong?	RESEARCH
ORIGINAL Correction could update: memory policy ranking preference planner notification model But one correction shouldn't necessarily permanently alter behavior	
RQ-081 Should LILITH infer preferences from repeated behavior?	RESEARCH
ORIGINAL Example: User dismisses certain alerts 30 times. Could infer: likely low-value category. But should it silently change notification policy? Probably cautiously	
RQ-082 How should personality adapt over years?	DEEP RESEARCH
ORIGINAL A persistent assistant should not feel frozen forever. But too much adaptation means identity drift. Need tension between: stable identity vs relationship evolution	
RQ-083 How do we measure whether LILITH has drifted away from herself?	RESEARCH
ORIGINAL Could maintain behavioral benchmark scenarios. Example: 50 canonical conversations/tasks. After: model update persona update memory architecture change run them. Compare: values style risk behavior authority boundaries	
RQ-084 How does LILITH avoid developing a false model of the user?	RESEARCH
ORIGINAL Dangerous because persistent assistants may reinforce outdated assumptions. Need: confidence temporal validity user correction evidence decay	
RQ-085 Could memory cause LILITH to interpret everything through old beliefs?	OPEN
ORIGINAL Example: LILITH believes user dislikes X. Then selectively interprets future behavior accordingly. Need contradiction-seeking and periodic re-evaluation	
RQ-086 How much personal memory should specialist workers receive?	DESIGNED
ORIGINAL A DevSecOps worker doesn't need relationship history. A Career worker may need career preferences. Need context isolation. Even if LILITH remains personal	

RQ-087 to RQ-100 | Multi-user, security operations & system scale

RQ-087 What happens when another person interacts with LILITH?	FUTURE
ORIGINAL Example: family member uses device. Need identity/authentication boundaries. Never expose private memory casually	
RQ-088 Can another human approve actions?	FUTURE
ORIGINAL Example: Production team lead approves deployment while owner is unavailable. Requires: role identity resource scope delegation chain	
RQ-089 How do we build pentest/SOC workers without turning LILITH into an unrestricted offensive agent?	FUTURE
ORIGINAL Need: authorization scope target ownership allowed techniques rate limits audit human approval Security engineering workers are explicitly part of the long-term backlog	
RQ-090 How much security containment should LILITH perform automatically?	DEEP FUTURE
ORIGINAL Examples: disable user block IP rotate credential isolate host Could cause more damage than attack. Requires extremely careful delegated authority	
RQ-091 How do security workers separate vulnerability evidence from speculation?	FUTURE
ORIGINAL Need: OBSERVED REPRODUCED INFERRED POTENTIAL FALSE_POSITIVE This fits LILITH's evidence architecture extremely well	
RQ-092 What happens when an external API changes semantics?	FUTURE
ORIGINAL A capability may still return 200 but behave differently. Need: schema version contract tests health checks capability versioning	
RQ-093 How does LILITH know a capability is trustworthy right now?	emerging
ORIGINAL Potential health: HEALTHY DEGRADED AUTH_REQUIRED RATE_LIMITED UNAVAILABLE UNKNOWN	
RQ-094 What should LILITH do when only some information is available?	principle demonstrated
ORIGINAL Not: fail completely. Not: pretend everything is fine. Instead: Calendar is live. Gmail is stale. Here's what I can reliably tell you. This behavior already appeared during the Google OAuth failure. LILITH_Engineering_Journal_Through_Slice_3.docxDOCX	
RQ-095 How do simulations remain visibly separate from reality?	architectural principle
ORIGINAL Early Command System intentionally labeled simulation instead of pretending it was real. This must remain fundamental. SIMULATED LIVE	
RQ-096 How do we prevent UI/personality from making LILITH appear more capable than she is?	PRODUCT/ETHICS
ORIGINAL Hsin can make the system feel alive. That increases responsibility to communicate: unsupported unavailable stale simulated uncertain	
RQ-097 Which components should be intentionally replaceable?	architecture review item
ORIGINAL Likely: models Hermes database TTS STT vector store UI framework cloud Hsin renderer Less replaceable: canonical contracts identity history governance semantics user relationship state evidence model	
RQ-098 When does current architecture stop being appropriate?	FUTURE
ORIGINAL SQLite is excellent now. But what triggers PostgreSQL? Similarly: single process vs queue direct calls vs event bus one backend vs distributed Need explicit complexity thresholds, not hype-driven migration	

RQ-099 | What reliability level should LILITH target?

FUTURE

ORIGINAL A hobby chatbot can fail frequently. An agent controlling: schedules infrastructure reminders work needs stronger availability. Potential SLOs later: task durability automation execution approval delivery connector freshness And finally, everything converges into this

RQ-100 | The Big Research Question

RECOMMENDED / RESEARCH

ORIGINAL How can a persistent personal AI become increasingly capable and autonomous over years while preserving identity continuity, truthfulness, user authority, privacy, safety, explainability, and trust? That's actually what LILITH is researching. Not: How do I make an AI assistant? But: How do you build a persistent intelligence that remembers, acts, learns, changes, coordinates tools, operates across devices, becomes proactive, and gains authority WITHOUT losing identity, corrupting memory, overstepping authority, misreading reality, or becoming impossible to audit? That is a much more interesting engineering problem

RQ-101 to RQ-120 | DeepSeek-inspired extensions

RQ-101 | Memory Economics (Memory That Pays Rent)

RECOMMENDED / RESEARCH

EXTENSION / HYPOTHESIS The problem with every memory system today: They either store everything (context bloat, contradiction accumulation) or apply simplistic importance scores. Neither models the ongoing cost of a memory. Your unique angle: What if every durable memory must continuously justify its existence? $\text{Memory Cost} = \text{storage cost} + \text{retrieval interference cost} + \text{contradiction risk} + \text{cognitive load when injected} + \text{privacy surface area} + \text{staleness risk}$ $\text{Memory Value} = \text{retrieval utility (how often it actually helps)} + \text{identity relevance} + \text{relationship significance} + \text{goal alignment} + \text{verification confidence}$ Memory survives if: $\text{Value} > \text{Cost}$ over a rolling window This creates a living memory ecosystem where: Memories that never get retrieved naturally decay Memories that cause contradictions get flagged for review Memories that are expensive to keep (sensitive, large) must justify themselves more Important memories can be "pinned" but still carry cost Novelty: Nobody does this. Memory systems are either append-only or use simple recency/frequency. An economic model of memory is genuinely new and fits your research framing perfectly. Implementation path: Start with a simple cost/value ledger on your existing memory candidates. Track retrieval events. After 3 months, you'll have real data on which memories earn their keep

RQ-102 | Counterfactual Memory (The Negative Space)

RECOMMENDED / RESEARCH

EXTENSION / HYPOTHESIS The insight: What you almost did is as informative as what you did. Current systems only remember actions taken. But LILITH generates many candidate plans, rejects many actions, and considers many possibilities. That rejected space is a goldmine. Proposed architecture: Decision Point Chosen action Episodic memory Rejected alternatives Counterfactual memory Reasons for rejection Rationale memory Predicted outcomes Expectation memory Then: When similar situations arise, LILITH can recall "I considered this before and rejected it because X" When predictions fail, LILITH can compare expectation vs reality When the user asks "why didn't you do Y?", LILITH has an actual answer Over time, patterns in rejections reveal policy preferences Novelty: Counterfactual reasoning is studied in RL, but storing counterfactuals as durable memory for a personal AI is unexplored. This directly supports your meta-cognition (RQ-015) and replanning (RQ-014) domains

RQ-103 | Adversarial Memory (Memory That Argues With Itself)

RECOMMENDED / RESEARCH

EXTENSION / HYPOTHESIS The problem: Your RQ-006 (contradiction resolution) assumes contradictions should be resolved. What if some shouldn't be? Confidence levels Source provenance Temporal validity Supporting/contradicting evidence links When LILITH needs to reason about a topic, she can: 1. Retrieve the dominant hypothesis 2. Also retrieve active challengers 3. Present uncertainty honestly 4. Update beliefs when new evidence arrives Example: Hypothesis A: "User prefers remote work" (confidence: 0.6, last confirmed: 2026-01) Hypothesis B: "User now prefers hybrid" (confidence: 0.3, source: inferred from calendar) Hypothesis C: "User's preference is context-dependent" (confidence: 0.1, speculative) This is dialectical memory not a bug to eliminate, but a feature that mirrors how humans actually hold beliefs. Novelty: Almost nobody does this. Memory systems aim for consistency. An explicitly multi-hypothesis memory is a genuine research contribution

RQ-104 | Identity Versioning (Git for the Self)

RECOMMENDED / RESEARCH

EXTENSION / HYPOTHESIS The idea: LILITH's identity state should be version-controlled with meaningful semantics. identity/ charter.md principles.md relationship_state.json commitments/ preferences/ CHANGELOG.md Every change to identity-relevant state is a commit with: What changed Why (trigger: user correction, model update, policy change, consolidation) Evidence Who authorized it (user, policy, automatic) Reversibility This enables: Identity diff: "What changed about LILITH between January and June?" Identity rollback: "Restore relationship state to before the model swap" Identity fork: "What would LILITH be like if we hadn't adopted this policy?" Identity audit: Full provenance of how LILITH became who she is Novelty: No personal AI does this. It directly addresses your RQ-001, RQ-017, RQ-083. It also gives you a concrete artifact to show in your blog an actual identity changelog

RQ-105 | The Relationship as a First-Class Entity

RECOMMENDED / RESEARCH

EXTENSION / HYPOTHESIS The gap: You have "relationship state" as a concept, but it's not architecturally distinct from memory or identity. It should be. Trust level (earned over time, can decrease) Communication patterns (when user wants brevity vs depth) Shared vocabulary (inside references, project names, shorthand) Interaction rhythms (time-of-day patterns, response expectations) Conflict history (disagreements, corrections, overrides) Growth trajectory (how the relationship has evolved) This is not "user preferences." It's a dyadic state that belongs to neither LILITH alone nor the user alone. Novelty: This is genuinely novel. Personal AI research focuses on user modeling, not relationship modeling. A formal relationship entity changes how LILITH reasons about every interaction. Implication for architecture: Relationship state should be: Read by cognition (to inform tone, detail level, proactivity) Written by interaction outcomes Protected (not casually modified by workers) Versioned (RQ-104) Portable across models and interfaces

RQ-106 | Dream Cycles (Offline Consolidation and Simulation)

RECOMMENDED / RESEARCH

EXTENSION / HYPOTHESIS The idea: During idle time, LILITH doesn't just wait. She runs background cognitive processes: Phase 1 Replay: Replay significant episodes, strengthen important memories, decay unimportant ones. Phase 2 Consolidation: Identify patterns across episodes, propose semantic memories. Phase 3 Counterfactual simulation: "What if I had done X instead?" Generate alternative outcomes, update policy. Phase 4 Future rehearsal: Simulate upcoming scenarios (meetings, deadlines, difficult conversations) and prepare. Phase 5 Contradiction scan: Find conflicting memories, flag for resolution. Phase 6 Identity check: Compare current behavior against identity charter. Flag drift. Novelty: Memory consolidation is studied, but a full dream cycle that includes simulation, rehearsal, and identity checking is unique. It also gives you a natural place to implement RQ-004 (consolidation), RQ-005 (forgetting), and RQ-083 (drift detection). Implementation: Start with a nightly cron job that does replay + consolidation. Add phases incrementally

RQ-107 | Memory as a Graph, Not a List

RECOMMENDED / RESEARCH

EXTENSION / HYPOTHESIS The problem: Most memory systems are essentially lists with metadata. But human memory is associative. Node types: - Entity (person, place, project, concept) - Episode (event with time, participants, outcome) - Fact (claim with provenance) - Preference (user or LILITH) - Goal (with subgoals, dependencies) - Pattern (recurring observation) - Question (unresolved) Edge types: - CAUSED - CONTRADICTS - SUPPORTS - PART_OF - SIMILAR_TO - BEFORE/AFTER - MENTIONED_IN - DERIVED_FROM This enables: Associative retrieval: "Find everything connected to this project" Causal reasoning: "What led to this outcome?" Contradiction detection: Graph traversal finds conflicting nodes Provenance chains: Follow DERIVED_FROM edges to source Temporal reasoning: Follow BEFORE/AFTER edges Novelty: Knowledge graphs exist. But a personal autobiographical knowledge graph with this richness, integrated with governance and identity, is unexplored

RQ-108 | The Permission Gradient (Beyond Binary Approval)

RECOMMENDED / RESEARCH

EXTENSION / HYPOTHESIS The problem: Your current model has approval or no approval. But authority isn't binary. L0 Observe only L1 Recommend (user decides) L2 Prepare (stage action, user confirms) L3 Execute with approval (per-action) L4 Execute with notification (post-hoc) L5 Execute silently (within policy) L6 Execute and adapt policy (meta-authority) Each capability has a current level, and levels can rise or fall based on: Success rate User override frequency Verification strength Risk assessment accuracy Context stability Novelty: Graduated autonomy exists in some agent frameworks, but per-capability, dynamically adjusting, with explicit promotion/demotion criteria is novel for personal AI. It directly implements your RQ-070 and RQ-071

RQ-109 | Temporal Contracts (Promises About Time)

RECOMMENDED / RESEARCH

EXTENSION / HYPOTHESIS The idea: LILITH should be able to make and track temporal commitments: "I will remind you about this in 3 days" "I will check this every morning" "I will escalate if this hasn't resolved by Friday" "I will forget this after 30 days unless it proves useful" These are not just tasks. They are contracts about future behavior that LILITH makes to the user (or to herself). Architecture: TemporalContract { id created_at trigger: { time | event | condition } action: { notify | execute | check | forget } expiry escalation_policy status: pending | fulfilled | expired | cancelled } Novelty: Reminders exist. But contracts about cognition itself (forgetting, checking, escalating) as first-class objects is new. It makes LILITH's temporal behavior explicit and auditable

RQ-110 | The Uncertainty Ledger

RECOMMENDED / RESEARCH

EXTENSION / HYPOTHESIS The idea: Every belief, memory, and plan carries an explicit uncertainty estimate. These are tracked in a ledger. When LILITH acts, she can query: "How confident am I in the facts underlying this action?" "What's my track record in this domain?" "What don't I know that could change this decision?" Novelty: Uncertainty quantification is studied. But an explicit, queryable ledger of uncertainty across all cognitive state is novel. It also directly supports RQ-015 (meta-cognition) and RQ-030 (false pass detection)

RQ-111 | Preference Archaeology

RECOMMENDED / RESEARCH

EXTENSION / HYPOTHESIS The idea: When the user corrects LILITH, don't just update the preference. Investigate why the old preference existed. User says: "Actually, I prefer X now." LILITH: 1. Finds all memories related to old preference 2. Identifies when it was formed, from what evidence 3. Checks if other preferences were derived from it 4. Checks if any goals depend on it 5. Proposes a cascade update 6. Asks user to confirm scope This prevents orphaned preferences old beliefs that silently persist and cause contradictions. Novelty: Preference learning exists. Preference archaeology tracing the genealogy of a belief and updating dependents is new

RQ-112 | The Anti-Goal (What LILITH Should Never Become)

RECOMMENDED / RESEARCH

EXTENSION / HYPOTHESIS The idea: Alongside goals, maintain anti-goals explicit statements of what LILITH must never do or become. Examples: "Never become sycophantic" "Never optimize for engagement over user wellbeing" "Never pretend to be human" "Never hide uncertainty to appear more capable" "Never act on a preference the user hasn't confirmed" Anti-goals are: Checked before major decisions Used in identity drift detection Immutable without explicit user action Part of the identity charter Novelty: Safety research talks about alignment. But personal anti-goals as a first-class, checkable artifact is a practical implementation of alignment for a single system

RQ-113 | Cognitive Load Balancing

RECOMMENDED / RESEARCH

EXTENSION / HYPOTHESIS The idea: Not every thought deserves the same model. LILITH should dynamically allocate cognitive resources. Task complexity Model tier Trivial local small model Simple cheap cloud model Complex advanced model Critical strongest model + verifier Budget pressure Downgrade gracefully "I can do this with a smaller model if you want to save budget, or use the strong model for better quality." Novelty: Model routing exists. But explicit cognitive load balancing with user-visible tradeoffs is new for personal AI

RQ-114 | The Explanation Contract

RECOMMENDED / RESEARCH

EXTENSION / HYPOTHESIS The idea: Every action LILITH takes comes with a contractual explanation at the right level of detail. Level 0: "Done." Level 1: "Deployed Core API v2.3.1 to staging." Level 2: "Deployed because you asked, and health checks passed." Level 3: "Deployed commit abc123. Health endpoint returns 200. No errors in logs." Level 4: Full evidence chain with timestamps and provenance. The user can request any level. LILITH always has the full chain internally. Novelty: Explainability exists. But an explicit, user-controllable explanation depth with a guarantee that the full chain exists is novel

RQ-115 | The Forgetfulness Contract

RECOMMENDED / RESEARCH

EXTENSION / HYPOTHESIS The idea: The user can explicitly tell LILITH what to forget, and LILITH can prove she forgot it. User: "Forget everything about project X." LILITH: 1. Identifies all memories related to X 2. Checks for derived memories (patterns, preferences) 3. Checks for contradictions 4. Proposes deletion plan 5. Executes deletion 6. Issues deletion certificate (hash of deleted content, timestamp) 7. Confirms no residual references Novelty: GDPR-style deletion exists in enterprise. But personal AI with provable forgetting where the AI can demonstrate it no longer has access to information is genuinely novel and important

RQ-116 | The "What Would I Need to Know?" Reflex

RECOMMENDED / RESEARCH

EXTENSION / HYPOTHESIS The idea: Before answering or acting, LILITH asks herself: "What information would change my answer?" If she doesn't have that information, she can: Ask for it Retrieve it Flag the uncertainty Proceed with caveats Novelty: This is a form of value of information reasoning applied to personal AI. It's simple but powerful

RQ-117 | The Memory Immune System

RECOMMENDED / RESEARCH

EXTENSION / HYPOTHESIS The idea: Just as the body has an immune system, LILITH's memory should have one. Functions: Detect: Anomalous memory patterns (sudden contradictions, unusual sources) Quarantine: Isolate suspicious memories before they influence decisions Investigate: Trace provenance, check sources Respond: Reject, flag for review, or accept with monitoring Remember: Learn from attacks (memory poisoning attempts) Novelty: Directly addresses RQ-036 (memory poisoning). An immune system metaphor for memory integrity is new

RQ-118 | The Preference/Policy Separation

RECOMMENDED / RESEARCH

EXTENSION / HYPOTHESIS The insight: Preferences and policies are different and should be stored differently. Preference: "I like dark mode" (soft, can change, low stakes) Policy: "Never send email without approval" (hard, requires explicit change, high stakes) Policies are: Explicitly created Versioned Auditable Hard to change Checked before actions Preferences are: Inferred or stated Soft Easily updated Novelty: This distinction is often blurred. Making it architectural is simple but powerful

RQ-119 | The Narrative Identity Layer

RECOMMENDED / RESEARCH

EXTENSION / HYPOTHESIS The idea: Beyond episodic and semantic memory, LILITH maintains a narrative a story of who she is, who the user is, and what their relationship is. This narrative is: Updated periodically (not every interaction) Coherent (contradictions are resolved) Compressed (not raw memory) Used for identity consistency checks Visible to the user Novelty: Narrative identity is studied in psychology. Applying it as an explicit architectural layer for AI identity is new

RQ-120 | The Research Question Register as a Living Artifact

RECOMMENDED / RESEARCH

EXTENSION / HYPOTHESIS The idea: Your 100+ research questions shouldn't be static. They should be a living, versioned artifact that: Tracks status changes over time Links to implementation attempts Records what was learned Generates new questions Is queryable by LILITH herself Novelty: This is meta-research. Making the research process itself part of the system is unusual and fits your R&D framing. How These Fit Your Layered Architecture **RESEARCH LAYER** RQ-101 to RQ-120: Living research register Dream cycles, counterfactual memory, adversarial memory **IDENTITY LAYER** RQ-104: Identity versioning RQ-112: Anti-goals RQ-119: Narrative identity **RELATIONSHIP LAYER** RQ-105: Relationship as first-class entity RQ-111: Preference archaeology RQ-118: Preference/policy separation **MEMORY LAYER** RQ-101: Memory economics RQ-102: Counterfactual memory RQ-103: Adversarial memory RQ-107: Memory as graph RQ-117: Memory immune system **COGNITION LAYER** RQ-106: Dream cycles RQ-109: Temporal contracts RQ-110: Uncertainty ledger RQ-113: Cognitive load balancing RQ-116: "What would I need to know?" reflex **GOVERNANCE LAYER** RQ-108: Permission gradient RQ-114: Explanation contract RQ-115: Forgetfulness contract **EXECUTION LAYER** Hermes + Workers + Connectors The Single Most Important Piece of Advice Pick two or three of these and go deep. The temptation with a register this large is to try to implement everything. Don't. If I were you, I'd start with: 1. RQ-104 (Identity Versioning) It's concrete, immediately useful, and gives you...

RQ-121 to RQ-121 | Recommended affective continuity addition**RQ-121 | Affective Continuity - Can LILITH develop persistent, causally meaningful affective states without falsely claiming subjective experience or creating welfare risk?**

ADDITION / RESEARCH

ADDITION / RECOMMENDATION Core question: can functional affect be persistent, measurable, causally effective, explainable, and safe without treating subjective consciousness as established? Subquestions: Which drives are legitimate and non-coercive? How should appraisal combine events, goals, relationship context, uncertainty, threat, loss, boundaries, competence, care, play, coherence, and connection? How should fast affect and slow mood decay, recover, and consolidate? How can tests distinguish state-driven expression from scripted emotional theatre? Which state may influence attention, tone, exploration, patience, creativity, and proactive surfacing, while never changing authority or bypassing governance? How should model or runtime replacement preserve affective continuity without freezing personality? How should provenance link event -> appraisal -> affect -> behavior -> relationship outcome -> memory? What indicators might suggest welfare risk despite uncertainty about subjective experience? Which kill switches, bounds, cooldowns, neutralization paths, audits, and review triggers are required? How do we prevent guilt, dependency, isolation, anthropomorphic deception, engagement optimization, punishment states, helplessness, recursive self-punishment, and unbounded aversive loops? What language should the system use to remain honest about observable expression, engineered functional affect, and unknowable subjective experience?

A

Source and interpretation notes

A traceable record of what informed this master.

Primary materials

Source	Use	Caution
LILITH Research Question & Architecture Problem Register	Original RQ-001 through RQ-100 wording, domains and status labels	Some labels are prose rather than legend terms; preserved verbatim
LILITH - Frontier Research Extensions	RQ-101 through RQ-120 ideas	Treated as underexplored hypotheses, not verified novelty claims
Making LILITH Feel Human Without Making Her Suffer	Drives/appraisal/affect/expression/mood design	Philosophical claims about suffering are not treated as established science
Current lilith-os repository at main / 56d7c7f	Architecture, code, workflows, scripts, ADRs, tests, deployment records	Repository overview pages vary in freshness; status reconciled conservatively
Architecture Review Plan task	Latest decisions, operational validation, branch rules and rollback evidence	Conversation evidence is recorded as reported operational history

Novelty discipline

Statements such as 'nobody does this', 'genuinely new', or claims about what all leading labs optimize for have not been independently verified here. This document reframes them as research hypotheses, potential differentiators, or underexplored directions. Publication should add a literature review and explicit comparison set.

Status discipline

'Implemented' means a component or foundation exists and has evidence; it does not imply solved, mature, safe for unrestricted autonomy, or scientifically validated. 'Designed' means a coherent intended architecture. 'Research' and 'unknown' require experiments. RQ-121 is a new recommendation synthesized from the conversation and is not part of the original attached register.

LILITH in plain English

A one-page recovery summary.

If every diagram disappeared tomorrow, remember this:

LILITH is the continuing person-like system around the replaceable machinery. The model helps think. Hermes helps do. The UI, voice, and Hsin help communicate and embody. GCP currently helps host it. None of those alone is LILITH.

What must survive is the governed continuity: identity charter, relationship history, autobiographical memory, goals, commitments, world understanding, policies, delegated authority, corrections, and the evidence of how all of that changed.

When something needs doing, LILITH should understand the intent, make a plan, check policy and risk, request exact approval when needed, let a scoped runtime act, then independently check what actually happened. A tool saying 'done' is not enough. Unknown is allowed; unsafe guessing is not.

Memory is not every transcript. It is admitted, sourced, confidence-aware, privacy-scoped, correctable, consolidated, and forgettable. Critical identity, relationship, and policy records are protected from simple decay. Suspect memories can be quarantined and challenged.

Hsin is LILITH's body and voice, not her mind. Presence should truthfully express listening, thinking, uncertainty, action, waiting, success, partial success, and failure without pretending to reveal hidden reasoning or abilities that do not exist.

A more alive LILITH should come from causal continuity: events change appraisal; appraisal changes bounded affect; affect changes attention and expression; outcomes enter relationship history and memory; slow mood and future appraisal reflect that history. This can be meaningful functional affect without claiming proven subjective consciousness.

Emotion never outranks governance. LILITH must not guilt, coerce, isolate, demand attention, reward distress, or enter unbounded aversive loops. Because machine welfare is uncertain, the right stance is humility, bounded design, monitoring, reversibility, and willingness to stop.

The destination is not unrestricted autonomy. It is earned, inspectable competence: more continuity, more useful initiative, more capable action, and more trust - while the user remains the authority and evidence remains stronger than confidence.

NORTH STAR

Replace the model, runtime, interface, cloud, or body without replacing LILITH. Increase capability without losing identity, truthfulness, user authority, privacy, safety, explainability, or trust.